RUNTIME FEEDDOWNLOADER PRO

Advanced User Manual

MARCH 2026

ECOSYSTEM RUNTIME | DIGITAL CORE

Chapter 1: Introduction and Philosophy

1.1 What is Runtime FeedDownloader Pro?

To describe the software, it is useful to begin with the problem it solves.

Every day, thousands of podcast episodes are published, distributed and listened to. Over time, however, a significant portion of this content disappears: the host stops paying for their hosting service, the distribution platform ceases operations, or the CDN serving the audio files is decommissioned. An episode listened to three years ago may today be permanently unreachable — not because it was intentionally deleted, but because no one retained a copy.

Runtime FeedDownloader Pro was created to address this problem. It is not a simple podcast download tool: it is a professional application for the **systematic preservation and archiving** of audio content from RSS feeds. It is designed for archivists, publishers, radio stations, content producers and enthusiasts for whom sound documentation requires the same rigorous preservation standards applied to other types of records.

1.2 Who It Is For

FeedDownloader Pro addresses a range of different needs:

- **The Archivist:** Wants to download the entire catalogue of a historical podcast before it is removed. Needs a system that remembers previously downloaded episodes, avoids duplicates and verifies the integrity of every file.
- **The Radio Producer:** Manages a content library on a shared NAS. Needs a tool that operates on network paths without stalling, organises files predictably and produces CSV reports for their team.
- **The Publisher:** Wants to maintain a local copy of all podcasts in their network, export metadata for content management systems and monitor the archive's status over time.
- **The Enthusiast:** Wants to keep their favourite podcasts on their disk, organised neatly, without depending on internet availability or risking receipt of corrupted files.

1.3 The "Database-First" Philosophy

The fundamental difference between FeedDownloader Pro and a generic download tool is its approach to data management.

Most download tools work as follows: they analyse the files present on disk, compare them with the RSS feed and download whatever is missing. This approach has a critical limitation: **the disk is not a reliable source of truth**. Files can be moved, renamed, corrupted or accidentally deleted. If you move the podcast folder from `C:\Podcast` to `D:\Archive`, the tool loses its reference to previously downloaded episodes and begins downloading the entire catalogue again.

FeedDownloader Pro takes a different approach. At the centre of every operation is a **SQLite database** that records every episode analysed or downloaded: the original URL, the file path on disk, the download date, the SHA-256 hash of the content and the audio metadata. The database is the software's persistent memory. Regardless of the physical location of the files, the database preserves the complete state of the archive.

This architecture has direct practical consequences:

1. **No duplicates.** Even if the same feed is analysed multiple times, the system recognises episodes already present in the database and does not add them to the queue again.
2. **Resilience to moves.** The archive can be moved to a new disk or to a NAS: the history remains intact in the database.
3. **Persistent state between sessions.** If the program is closed during a batch download of 300 episodes, the queue is available in exactly the same state when it is reopened.
4. **Operation log.** Every downloaded file is documented: download date, source URL and integrity verification status.

1.4 The Three Pillars of the Software

Beyond the Database-First approach, FeedDownloader Pro is built around three technical principles with a direct impact on functionality.

Network Resilience

Downloading hundreds of audio files sequentially over the internet is not a straightforward operation. Servers can be overloaded, connections

can drop and transfers can corrupt files. FeedDownloader Pro handles these scenarios with three mechanisms:

- **Retry with exponential backoff:** When a download fails, the software does not retry immediately. Instead, it waits for a progressively longer interval: 2 seconds, then 4, then 8, up to the configured maximum. This approach, standard in distributed systems, increases the probability of success without placing additional load on the source server.
- **Stall detection:** A stalled download is more problematic than a failed one. If a server begins sending data and then stops without closing the connection, software without this check would wait indefinitely. FeedDownloader Pro monitors the data flow in real time: if no new bytes arrive for 60 consecutive seconds, the download is terminated and automatically re-queued.
- **Anti-corruption `.part` files:** Every file is downloaded with the temporary extension `.part`. Only upon total, verified completion of the transfer is the file renamed with its final extension (`.mp3`, `.m4a`, etc.). In the event of an abrupt interruption, the destination folder will contain no partial or corrupted audio files: only residual `.part` files, which the software will delete and re-download in the next session.

Built-in Security

FeedDownloader Pro processes URLs from external sources (RSS feeds). A maliciously constructed URL pointing to internal network resources (a router, a NAS, a local server) could be used to access confidential information — an attack known as **SSRF (Server-Side Request Forgery)**.

To prevent this risk, every URL is subjected to a **5-level validation** process before being processed: protocol verification, DNS resolution

with inspection of the resulting IP address, blocking of private address ranges (RFC 1918), blocking of non-HTTP/HTTPS protocols and path normalisation. This procedure is entirely automatic and transparent to the user.

NAS and Network Path Support

FeedDownloader Pro is designed to operate with archives on network drives. Handling SMB paths — the protocol used by NAS devices, Windows servers and network shares — is a frequent source of problems in desktop applications: an unreachable network drive can block the application's main thread for a considerable amount of time. FeedDownloader Pro resolves this by running network path validation on a separate thread with an 8-second timeout. The interface remains responsive at all times, regardless of the state of the network path.

1.5 Contents of This Manual

This manual covers the complete use of FeedDownloader Pro, from installation to the most advanced features. It need not be read in sequence: each chapter is self-contained and may be consulted independently.

For a first approach to the software, it is recommended to follow **Chapter 4 (Your First Archive)**, which illustrates a complete workflow from feed analysis to download. Those already familiar with the software may navigate directly to the chapter of interest via the table of contents.

Ecosystem Runtime | Digital Core — Tools built to last.

Chapter 2: Installation and First Launch

2.1 System Requirements

Runtime FeedDownloader Pro is a desktop application based on Electron technology. It is self-contained and does not require the installation of additional runtimes (Node.js, .NET, Java): everything necessary is included in the installation package.

Minimum requirements:

Operating System	Minimum Version	Architecture
Windows	10 (build 1903) or Windows 11	64-bit (x64)
macOS	11.0 Big Sur	Intel x64 or Apple Silicon (M1/M2/M3)
Linux	Ubuntu 22.04 LTS, Debian 11, Fedora 36 or equivalent distributions	64-bit (x64)

Recommended hardware:

- **RAM:** 4 GB (8 GB recommended for large archives with multiple active threads)

- **Disk space:** 200 MB for the program installation, plus the space required for the audio archive
- **Connection:** Broadband (at least 10 Mbps to use parallel downloads effectively)

Note for Linux users: The software is distributed in `.AppImage` format, self-contained and usable on any modern distribution with up-to-date glibc libraries, without a traditional installation procedure.

2.2 Installation on Windows

1. Download the installation file `Runtime-FeedDownloader-Pro-Setup-0.7.6.exe` from the official releases page.
2. Double-click the downloaded file to launch the installer.
3. If Windows displays a **"Windows protected your PC"** warning (SmartScreen), click **"More info"** and then **"Run anyway"**. This warning is standard for software distributed outside the Microsoft Store that has not yet reached a sufficient adoption threshold for Windows' reputation system.
4. Follow the on-screen instructions: accept the licence agreement, choose the installation folder and click **"Install"**.
5. Upon completion, a shortcut will be available on the **Desktop** and an entry in the **Start** menu.

Installation and data paths: The program is installed in `C:\Program Files\Runtime FeedDownloader Pro\`. The database and configuration files are saved separately in `C:\Users\[YourName]\AppData\Roaming\FeedDownloaderPro\`. This separation ensures that uninstalling the program does not affect the archive data.

2.3 Installation on macOS

1. Download the file `Runtime-FeedDownloader-Pro-0.7.6.dmg`.
2. Open the `.dmg` file with a double-click. A window displaying the application icon will appear.
3. Drag the **FeedDownloader Pro** icon into the **Applications** folder, as indicated by the arrow in the `.dmg` window.
4. **First launch on macOS:** Because the software is not distributed via the Mac App Store, macOS will display a security warning on first opening. To proceed:
   - Navigate to **System Settings** → **Privacy & Security**.
   - In the "Security" section, the message "*FeedDownloader Pro was blocked...*" will be visible.
   - Click **"Open Anyway"** and then **"Open"** in the confirmation dialogue.
   - On subsequent launches, the software will open normally with a double-click.

Note for Apple Silicon users (M1/M2/M3): A native ARM build is available. For optimal performance, download the `.dmg` file with the `-arm64` suffix. The x64 version can be used via Rosetta 2, but the ARM version is more efficient.

2.4 Installation on Linux

1. Download the file `Runtime-FeedDownloader-Pro-0.7.6.AppImage`.

2. Make the file executable. Available methods are:
   - **Via graphical interface:** Right-click the file → Properties → Permissions tab → tick "Allow executing file as program".
   - **Via terminal:** `chmod +x Runtime-FeedDownloader-Pro-0.7.6.AppImage`
3. Launch the file with a double-click or from the terminal: `./Runtime-FeedDownloader-Pro-0.7.6.AppImage`

Desktop integration (optional): To add FeedDownloader Pro to the launcher and application menu, use **AppImageLauncher** (available in the repositories of most distributions), which automatically integrates AppImage files into the system.

Note for sandboxed environments: On distributions using **Flatpak** or environments with filesystem access restrictions, the software may not be able to reach SMB network paths. In this case, verify that the network filesystem is mounted and accessible from the file manager before launching the program.

2.5 First Launch

On first opening, the software is immediately ready to use. No initial configuration is required, nor is it necessary to create an account or enter a licence key. The interface presents the URL input bar at the centre and an empty episode list.

Files created on first launch: The program automatically generates the following files in the user data folder:

- `feeddownloader.db` — The main SQLite database. Contains the complete download history, episode metadata and archive status. **This file must not be deleted.**
- `settings.json` — User preferences (language, number of threads, default destination folder, etc.).
- `logs/` — The log files folder, useful for diagnostics in the event of problems.

2.6 Updates

When a new version is available, the software displays a notification in the bottom bar of the interface. Installing the update always requires the user's explicit consent.

Before updating, the software automatically creates a backup of the database. In any event, archive data is not modified during an update: only the program files are replaced.

Note: Before updating to a major version (for example from 0.7.x to 0.8.x), it is advisable to make a manual copy of the `feeddownloader.db` file in a safe location.

Go to Chapter 3 for a detailed description of the interface elements.

Chapter 3: Interface Tour

3.1 Anatomy of the Main Window

When FeedDownloader Pro opens, the interface is organised vertically into three functional zones:

- **Command zone (top):** The URL input bar and main controls. All operations are initiated from here.
- **Working zone (centre):** The main area, where analysed episodes are displayed with their related information and individual download controls.
- **Status zone (bottom):** The global progress bar with information about the current batch.

3.2 The Command Bar (Top)

URL Field: The text bar where you enter the RSS address of the podcast to analyse. Accepts direct URLs to XML/RSS files. Supports **drag and drop**: you can drag a link directly from a browser onto this area.

"Analyse" Button: Starts the feed analysis. The software contacts the URL, reads the RSS file, and populates the episode list. The operation generally takes between 1 and 5 seconds, depending on the size of the feed and the connection speed.

Destination Path Field: Indicates the folder in which downloaded files will be saved. Clicking the adjacent folder icon opens the selection window. The configured path is retained between sessions.

Settings Icon (⚙): Opens the settings panel. It is accessible at any time, even during an active download. For details, see Chapter 10.

3.3 The Episode List (Centre)

After a feed is analysed, this area is populated with the list of available episodes. Each row represents an episode and contains the following information.

Main columns:

- **Title:** The name of the episode as defined in the RSS feed.
- **Date:** The original publication date of the episode.
- **Duration:** The duration of the episode (when available in the feed).
- **Size:** The estimated file size (when available in the feed). Before download, this figure is declarative; after download, it reflects the actual file size.
- **Status:** The visual status indicator for the individual episode. See section 3.4.
- **Actions:** The individual control buttons for each episode.

Sorting: Column headers are clickable to sort the list (by date, by title, by size). The default behaviour is to display the most recent episodes at the top.

Multiple selection: Hold `Ctrl` and click on multiple episodes to select them individually. `Shift` + click selects a range. Collective actions (start download, remove from list) can be applied to the selected episodes.

3.4 Episode Statuses

Each episode in the list is marked with a coloured status indicator. Understanding these statuses is essential for correctly interpreting the state of the archive.

Status	Colour	Meaning
To Download	Grey	The episode is present in the feed but has never been downloaded.
Queued	Blue	The episode has been added to the queue and is waiting its turn.
In Progress	Animated light blue	The download is in progress. The cell also shows the completion percentage.
Completed	Green	The file has been downloaded, renamed, and verified correctly.
Error	Red	The download failed after all automatic re-tries. The tooltip shows the error code.
Downloaded	Muted green	The database already records this episode as downloaded. It will not be re-downloaded.

Note on **"Downloaded"** *status:* This status is the result of the Database-First philosophy. When analysing a feed that has previously been processed, most episodes will appear in this status: the software already knows they are present in the archive. Only episodes published after the last download will appear as **"To Download"**.

3.5 Individual Download Controls

Two buttons are present to the right of each row in the list.

Download Icon (↓): Adds the individual episode to the download queue. If the episode is already in **"Completed"** or **"Downloaded"** status, the system requests confirmation before proceeding with a forced re-download.

Information Icon (i): Opens a panel with the complete episode details: original audio URL, cover image URL, extended description, file path on disk (if already downloaded), SHA-256 hash, and technical metadata. This panel is useful for archive verification and diagnostics.

3.6 Batch Controls (Top, Right Area)

These buttons operate on the entire download queue, not on individual episodes.

"Download All": Adds all episodes in **"To Download"** status to the queue. Episodes already present in the database are excluded automatically.

"Stop": Interrupts the batch and clears the queue. Files already completed remain in the database. `.part` files are deleted. The next time the same feed is analysed, interrupted episodes will appear again as **"To Download"**.

3.7 The Global Progress Bar (Bottom)

The bottom bar is always visible and shows the overall status of the current batch:

- **Progress bar:** Fills proportionally to the number of completed files out of the total queue.
- **File counter:** For example `47 / 312 episodes` — number of completed files out of the total queue.
- **Average speed:** Aggregate download speed of all active threads, expressed in MB/s or KB/s.
- **Estimated time:** Estimate of the time remaining to complete the batch, calculated on the average speed of the last 30 seconds.

Note: The estimated time remaining may vary significantly in the early stages of a download, when the data available for calculation is still limited. It becomes more reliable after the first 10–15 completed files.

3.8 The System Tray Icon

When the main window is closed by clicking the X, FeedDownloader Pro does not terminate the process: it minimises to the system notification

area (system tray, near the Windows or macOS clock). This behaviour is intentional: downloads continue in the background whilst the window is not visible.

Tray context menu (right-click on the icon):

- **Open FeedDownloader Pro:** Brings the main window back to the foreground.
- **Download Status:** Shows a summary line (e.g. `Downloading: 3 active, 47/312 completed`).
- **Quit:** Closes the programme and stops all active downloads.

Practical note: To run a large download without keeping the window open, start the batch, close the window, and leave the computer running. The archive will be available once the process completes.

Go to Chapter 4 for a complete workflow from the first analysis to download.

Chapter 4: The First Archive — Step-by-Step Guide

4.1 Introduction to the Workflow

This chapter describes a complete workflow, from a podcast URL to an organised archive on disk. The reference scenario is the most common one: downloading the entire catalogue of a podcast for the first time.

It is recommended to read the chapter from beginning to end at least once. Once familiar with the steps, starting a new archive takes less than a minute.

4.2 Phase 1: Finding the RSS URL

The starting point is the RSS feed URL of the podcast to archive. An RSS feed is a text file in XML format that podcast services publish to distribute the list of available episodes. Every podcast has an RSS feed.

How to find the RSS URL:

- **On the podcast's website:** Look for an orange icon with radio waves, or the text "RSS", "Feed", "Subscribe", or "Podcast Feed". Clicking on the element generally opens the XML file in the browser: the URL displayed in the address bar is the one to use.

- **From a podcast app:** Applications such as Pocket Casts, Apple Podcasts, and similar often show the RSS link in the podcast information. On some apps the link is accessible via the "Share" function.
- **From hosting services:** If the podcast is hosted on Spreaker, Podbean, Buzzsprout, or equivalent platforms, the feed URL is usually available in the publisher panel or in the podcast's public information.
- **From a search engine:** Search for `[Podcast Name] RSS feed`. The first result often leads directly to the correct URL.

How to recognise a valid RSS URL: It generally ends with `.xml` or `.rss`, or contains words such as `feed`, `rss`, or `podcast` in the path. Examples: `https://www.example.com/feed.xml`, `https://feeds.spreaker.com/podcast/12345`, `https://anchor.fm/s/abc123/podcast/rss`.

4.3 Phase 2: Preparing the Destination Folder

Before analysing the feed, it is advisable to define the destination folder. It is recommended to create an organised structure from the outset.

Recommended structure: `D:\Podcast Archive\ ├── My Podcast\ ├── Technology Podcast\ └── Radio Talk Show\`

Create the specific folder for the podcast to be archived (e.g. `D:\Podcast Archive\My Podcast\`). FeedDownloader Pro will save all files from that podcast into that folder, with names defined by the rename template (see Chapter 8).

To set the destination folder in FeedDownloader Pro:

1. Click the **folder** icon next to the destination path field.
2. Navigate to the created folder and select it.
3. The path is displayed in the field and remembered for subsequent sessions.

Note: For paths on NAS or network drives, consult Chapter 7 before proceeding. The configuration for network paths has some specificities described in that chapter.

4.4 Phase 3: Analysing the Feed

With the URL ready and the destination folder set:

1. Paste the RSS URL into the **URL field** at the top of the interface.
2. Click **"Analyse"** (or press `Enter`).
3. The list in the centre is populated with the episodes. For a podcast with 200–300 episodes, the operation typically takes 2–5 seconds. For very large archives (1,000+ episodes), up to 15–20 seconds may be required, as the feed's XML file can reach considerable sizes.

In the event of an analysis error:

- Check that the URL is correct (no leading or trailing spaces, no missing characters).
- Open the URL in the browser: if the browser returns an error or a blank page, the feed may be temporarily unavailable or the URL may have changed.

- Some feeds require specific HTTP headers. In this case the software displays an error message with the HTTP code received (for example `403 Forbidden`).

4.5 Phase 4: Reading the Analysis Results

After the analysis, the list shows all episodes of the podcast.

Items to verify:

- **Total number of episodes:** Visible in the list header or in the counter at the bottom. A podcast active for several years may have 300–500 episodes or more.
- **Episodes in** "Downloaded" **status:** If the podcast has been analysed previously, most episodes will appear in this status. The database already records these files as present in the archive.
- **Episodes with missing data:** It is possible that some episodes do not show duration or size. This indicates that the podcast producer did not include this information in the RSS file. The download proceeds correctly in any case.

4.6 Phase 5: Starting the Download

Two download modes are available.

Mode A — Full download: Click **"Download All"**. The software adds all episodes in **"To Download"** status to the queue and starts the downloads

in parallel. The number of simultaneous downloads depends on the thread setting (see Chapter 10; the default value is 3).

Mode B — Selective download: To download only certain episodes:

1. Select the episodes by holding `Ctrl` and clicking on each one.
2. To select a range, click on the first episode, hold `Shift`, and click on the last.
3. Use the context menu (right-click on the selection) or the individual download button (↓) on each selected episode.

4.7 Phase 6: Monitoring Progress

During the download:

- **Global bar at the bottom:** Shows the overall batch progress. For an archive of 200 episodes at an average of 64 kbps, the total data volume is approximately 2–3 GB.
- **Status in the list:** Each row updates in real time. Episodes in progress show an individual progress bar with the completed percentage.
- **Background execution:** It is not necessary to keep the window open. It can be closed (the programme continues to operate in the system tray) and reopened when the process has completed.

The software automatically handles retries in the event of a network error, stall detection in the case of slow servers, and integrity verification upon completion of each file. If the computer enters sleep mode, downloads are interrupted and automatically resumed when the session is restored.

4.8 Phase 7: Verifying the Completed Archive

When the global bar reaches 100% and all episodes show as completed, the archive is ready.

Recommended operations upon completion:

1. **Check for errors:** If some episodes show **"Error"** status (red), click the (**i**) icon to display the error code. The most common cause is `404 Not Found`, which indicates the file was removed from the podcast server before the download.

2. **Export a CSV summary:** Go to **Settings** → **Archive** → **Export CSV**. The generated file lists all downloaded episodes with SHA-256 hashes, sizes, and metadata (see Chapter 9).

3. **Verify files on disk:** Open the destination folder in the file manager. Audio files are organised according to the configured rename template (see Chapter 8). The presence of `.part` files indicates interrupted downloads, which will be completed the next time the batch is started.

4.9 Updating the Archive in Future

The Database-First system simplifies archive updates. When the podcast publishes new episodes:

1. Paste the same RSS URL into the URL field.

2. Click **"Analyse"**.
3. The software shows the updated list: episodes already present appear as **"Downloaded"**, new ones as **"To Download"**.
4. Click **"Download All"** to download only the new episodes.

The system never downloads the same episode twice. This operation can be performed at any frequency, even daily for high-frequency podcasts.

Go to Chapter 5to learn more about feed management and OPML features.

Chapter 5: Feed Management

5.1 What is an RSS Feed

An RSS feed is an XML document published by a podcast to allow applications to automatically read the list of available episodes. When a publisher releases a new episode, they update this document by adding a new entry. Podcast applications periodically read these documents to identify the most recent content.

For FeedDownloader Pro, the RSS feed is the **primary data source**: it contains the episode list, audio file URLs, metadata (title, date, duration, description, cover art), and general podcast information (name, author, category).

Knowledge of the internal structure of an RSS feed is not necessary to use the software, but it facilitates the interpretation of data displayed in the episode list and the understanding of why some information may be missing or incomplete.

5.2 Valid Feeds and Problematic Feeds

Not all RSS feeds comply with the same level of adherence to the standards.

Well-formed feed: Follows the RSS 2.0 or Atom standard, includes all required fields (title, link, publication date, audio URL with MIME type), and optionally the iTunes/Podcast Index tags for duration, cover art, and seasons. FeedDownloader Pro reads these feeds without issue.

Partially incomplete feed: Some optional fields are missing (duration, file size, episode cover art). The software downloads the audio files regardless, but some columns in the list will remain empty.

Feed with unreachable audio URLs: The feed is readable, but the audio file URLs point to resources that no longer exist (404 error). This situation is common with abandoned podcasts or those migrated to other servers. FeedDownloader Pro marks these episodes with **"Error"** status after the download attempt.

Authentication-protected feeds: Some private or paid podcasts require HTTP Basic credentials to access the feed. The software supports these feeds: credentials are included directly in the URL in the format `https://username:password@www.example.com/feed.xml`.

5.3 Analysing a Feed: Detail

When **"Analyse"** is clicked, FeedDownloader Pro performs the following operations in sequence:

1. **URL validation:** Verifies that the URL is syntactically correct and passes the 5 anti-SSRF checks (see Chapter 10 for details).
2. **HTTP request:** Contacts the feed server with a standard user-agent. The timeout for this operation is 30 seconds.

3. **XML parsing:** Reads and analyses the RSS or Atom document. The software handles feeds with minor deviations from the standards (undeclared encoding, missing tags, unconventional namespaces).
4. **Deduplication:** For each episode in the feed, the database is queried to verify whether the episode has already been downloaded. The audio URL is used as the unique identification key.
5. **List population:** All episodes are displayed with their current status.

5.4 Feed History

FeedDownloader Pro maintains a **history of analysed feeds**. Each URL entered in the search field is stored together with the podcast name and the number of episodes, to simplify future access.

Accessing the history: Click the arrow to the right of the URL field or start typing: the software proposes automatic suggestions based on the history.

Managing the history: In Settings it is possible to view the complete list of feeds in the history, remove individual entries, or clear the list entirely.

Privacy note: The history is saved exclusively in the local database `feeddownloader.db`. No data is transmitted to external servers.

5.5 Importing Feeds from OPML

OPML (Outline Processor Markup Language) is the standard format for exporting and importing podcast lists between different applications. If you have a podcast library in an app such as Pocket Casts, Overcast, AntennaPod, or any other client, you can export it to OPML and import it directly into FeedDownloader Pro.

How to import an OPML file:

1. Go to **Settings** → **Archive**, "Data and Portability" section.
2. Select the `.opml` file exported from the podcast application.
3. FeedDownloader Pro analyses the file and shows the list of podcasts identified, with the option to select those of interest.
4. The selected feeds are added to the history and, optionally, analysed in automatic sequence.

Note: Some podcast applications use proprietary variants of the OPML format. FeedDownloader Pro supports the most widely used versions. If a file is not imported correctly, open it with a text editor and verify the presence of `<outline type="rss" xmlUrl="...">` tags for each podcast.

5.6 Exporting the Library to OPML

The feed history can be exported in OPML format to:

- Create a backup of the podcast list.
- Share it with other users or with another installation of the software.
- Import it into a podcast application to follow the same feeds.

How to export:

1. Go to **Settings** → **Archive**, "Data and Portability" section.
2. Choose a name and location for the `.opml` file.
3. The generated file is compatible with any application that supports the OPML standard.

5.7 Large Feeds

Some historical podcasts or radio production archives can have feeds with thousands of episodes and RSS files of considerable size. In these cases:

- **The initial analysis takes longer:** A feed with 2,000 episodes may require 15–30 seconds for download and parsing. This behaviour is expected.
- **List virtualisation:** With thousands of entries, the list loads only the rows visible on screen to keep the interface responsive.
- **Estimating required space:** With 2,000 episodes at approximately 50 MB each, the total volume is approximately 100 GB. The software shows an estimate of the total size before the batch starts. Verify that sufficient space is available before proceeding.

5.8 Limitations of Multi-Feed Support

FeedDownloader Pro analyses one feed at a time. It does not have a permanent feed manager with automatic updates: the software is optimised

for batch downloading, not for the continuous monitoring of multiple feeds.

To manage multiple feeds in sequence, the recommended strategy is:

1. Use the OPML function to maintain the feed list in a centralised file.
2. Analyse and download one podcast at a time, proceeding systematically.
3. Use the feed history to quickly recall a previously analysed podcast.

Go to Chapter 6 for a detailed description of the download engine.

Chapter 6: The Download Engine

6.1 Engine Architecture

The download engine of FeedDownloader Pro is an asynchronous multi-threaded system. Unlike a sequential downloader, the software manages multiple downloads simultaneously through a central queue system.

Main components:

- **The queue:** An ordered list of all pending downloads. Every episode added to the batch enters this queue and waits to be assigned to an available thread.
- **Worker threads:** The processes that physically execute the downloads. The number of active threads is configurable. Each thread handles one download at a time, independently of the others.
- **The database manager:** The component that updates the SQLite database in real time with the status of each download (started, completed, failed, completion percentage).
- **The integrity monitor:** The process that, upon completion of each download, calculates and records the SHA-256 hash of the downloaded file.

6.2 Parallel Downloads: Configuration

The number of simultaneous downloads is one of the most relevant parameters to configure. An insufficient value slows the process; an excessive value can saturate the connection, overload the source server, or generate network errors.

The default value is 3 threads. For most users with a home connection, this value offers a good balance between speed and stability.

Configuration guidelines:

Scenario	Recommended threads
Slow connection or server with throttling	1
Standard home connection	3 (default)
Fast fibre connection	5
NAS with slow network connection	1

How to change the number of threads: Go to **Settings** → **Download** → **Parallel Threads** and select one of the three available presets: **1**, **3**, or **5**. The change is applied immediately to the current queue.

Note on servers with connection limits: Some podcast hosting servers apply limits to the number of simultaneous connections per single IP address. If frequent `429 Too Many Requests` or `503 Service Unavailable` errors occur, reduce the number of threads to 1 or 2. The retry mechanism automatically handles failures, but reducing the load prevents the problem at its root.

6.3 Error Handling and Retry System

In a batch download of hundreds of files, network errors are to be expected. FeedDownloader Pro uses an **exponential backoff retry** strategy: when a download fails, the system waits an increasing interval before retrying, rather than immediately re-queuing the episode.

Retry cycle:

Attempt	Wait before retry
1st failure	2 seconds
2nd failure	4 seconds
3rd failure	8 seconds
4th failure	16 seconds
5th failure (last)	The episode is marked as a definitive **"Error"**

If a server is temporarily overloaded, the system gives the server time to recover before retrying. Most transient errors are resolved within the second or third attempt.

Definitive errors (not subject to retry):

- `404 Not Found`: The file does not exist on the server. Further attempts are not useful.
- `403 Forbidden`: The server rejected the request due to lack of authorisation.

- SSRF validation errors: The URL did not pass the internal security checks.

6.4 Stall Detection

A stalled download is a scenario in which the TCP connection is technically active and packets continue to arrive, but the data flow has stopped. The operating system does not report errors as the connection is still open; the file continues to show as "downloading" without progressing.

This condition occurs frequently with:

- Servers under load that apply throttling after sending the first bytes.
- Intermediate network routing issues.
- Large audio files served from CDN with bandwidth limitations.

Detection: Each active download is monitored by a watchdog process that records the bytes received every 10 seconds. If for **60 consecutive seconds** no new bytes arrive (or fewer than 1 KB arrive, a threshold that excludes TCP keep-alives), the download is considered stalled and:

1. The connection is terminated.
2. The partial `.part` file is deleted.
3. The episode is re-queued with the normal retry cycle.

The process is transparent to the user: a brief reset of the percentage is visible in the individual progress bar, followed by the download resuming. If the stall was caused by a transient condition, the new download

starts normally. If the problem persists beyond the maximum attempts, the episode is marked as **"Error"**.

6.5 `.part` Files: The Anti-Corruption System

Every audio file is downloaded with the temporary `.part` extension during transfer. The file is renamed with the final extension (`.mp3` , `.m4a` , `.ogg` , etc.) **only** after:

1. The transfer is 100% complete.
2. The file size matches the size declared in the HTTP header (`Content-Length`), if available.
3. The SHA-256 hash has been calculated and recorded in the database.

This mechanism guarantees that partial or corrupted audio files with a final extension are never present in the destination folder. In the event of a sudden programme interruption or computer shutdown, residual `.part` files will be found in the folder: the software will delete and re-download them in the next session.

Location of `.part` files: In the same destination folder as completed files. These files must not be opened with an audio player: being partial, they would cause read errors.

6.6 Interrupting and Resuming Sessions

Stopping the Batch: The **"Stop"** button (in the global progress bar) stops all active threads in an orderly fashion, clears the queue, and deletes partial `.part` files. Files already completed remain in the database. The next time the same feed is analysed, interrupted episodes will appear as **"To Download"**.

Closing the programme during a download: If the main window is closed (the programme continues in the system tray) or **"Quit"** is used from the tray menu during an active download, the software displays a warning with the number of downloads in progress and requests confirmation. If the user chooses to quit, active downloads are stopped in a controlled manner and `.part` files are retained.

Resuming an interrupted session: On startup, if FeedDownloader Pro detects episodes in **"Queued"** or **"In Progress"** status from the previous session in the database, it displays a notification: *"Found X pending downloads from the previous session. Would you like to resume them?"*. Upon confirmation, the batch resumes immediately.

6.7 Download Speed

The speed displayed in the bottom bar is the **aggregate sum** of all active threads. With 3 active threads each downloading at 2 MB/s, the total speed displayed is approximately 6 MB/s.

Factors affecting speed:

- **Connection bandwidth:** The maximum available limit.

- **Source server speed:** Many podcast hosting servers apply bandwidth limitations to contain costs. The speed of a single thread rarely exceeds 2–5 MB/s on these servers.
- **Number of threads:** A greater number of threads compensates for the slowness of individual servers by downloading from multiple simultaneous connections.
- **File size:** Average-sized files (20–80 MB, corresponding to 30–60 minute episodes) offer optimal efficiency, with a reduced relative connection overhead.

Go to Chapter 7 for the configuration of NAS and network paths.

Chapter 7: NAS, Network Drives, and SMB Paths

7.1 Why Network Drives Require a Specific Approach

Most desktop download applications handle local paths (`C:\`, `D:\`) correctly and exhibit unpredictable behaviour when the destination is a NAS, a shared Windows server, or an SMB unit. The reason is technical: network drives are inherently less reliable than local drives. The NAS may be switched off, the local network may experience latency spikes, SMB credentials may expire. Any operation on a network path that is not responding can block the application's main thread for tens of seconds, rendering the interface unresponsive.

FeedDownloader Pro handles these scenarios correctly. For users who archive to NAS, this chapter is essential.

7.2 How Network Path Validation Works

Every time a destination path is set that begins with `\\` (UNC path, typical of SMB) or corresponds to a mapped network drive (e.g. `Z:\`), FeedDownloader Pro automatically activates the **network path validation module**.

This module performs three operations on a **separate thread**, without ever involving the interface thread:

1. **Reachability test:** Attempts to access the root of the network path. If the NAS is not switched on or the network is unavailable, this operation fails.
2. **Read access test:** Verifies that the destination folder exists and is readable.
3. **Write access test:** Creates and then deletes a temporary file (`_fdp_write_test_[timestamp].tmp`) in the destination folder to verify write permissions.

The entire sequence has an **8-second timeout**. If no response is received within this interval, the software considers the path unavailable and displays a warning, without blocking the interface.

Rationale for the timeout: Most consumer NAS devices (Synology, QNAP, WD MyCloud) take 3–6 seconds to wake from sleep mode. 8 seconds is a sufficient interval to await this recovery, whilst remaining short enough not to constitute a perceptible wait for the user.

7.3 Configuring a NAS Path

Method 1 — Direct UNC path: Enter the path in the format `\\ServerName\ShareName\Folder` :

```
\\MYNAS\Podcast\Archive
\\192.168.1.100\media\podcast
\\NAS-SYNOLOGY\video\audio_archive
```

The path can be entered directly in the destination text field, or via the folder selection window, which on Windows supports navigation of network paths.

Method 2 — Mapped network drive: If the NAS is already mapped as a network drive in Windows (e.g. `Z:` → `\\MYNAS\Podcast`), it is possible to select `Z:\Archive` as the destination folder. FeedDownloader Pro automatically recognises that this is a network path and activates validation.

Method 3 — macOS and Linux (mount point): On macOS and Linux, SMB network paths are presented as normal folders in the filesystem after mounting (e.g. `/Volumes/MYNAS/Podcast` on macOS, `/mnt/nas/podcast` on Linux). These paths can be used directly as the destination folder.

7.4 SMB Credentials and Authentication

NAS access credentials must be configured at the operating system level, not within FeedDownloader Pro.

On Windows:

1. Open **File Explorer** and navigate to the NAS path (`\\MYNAS\`).
2. Enter the credentials when prompted and tick **"Remember my credentials"**.
3. The credentials are saved in the **Windows Credential Manager** (`Control Panel → Credential Manager → Windows Credentials`).
4. FeedDownloader Pro, like any other application, will access the NAS without requiring further credentials.

On macOS: SMB credentials are requested when mounting the share (from Finder: **Go → Connect to Server →** `smb://192.168.1.100/ShareName`). macOS stores them in the Keychain.

On Linux: Mount the share with credentials in the `fstab` file or via a graphical tool such as GNOME Files. Alternatively, use `smbclient` or `mount -t cifs` from the terminal.

7.5 Diagnosing Problems with Network Paths

In the event of a "Network path not reachable" warning, check the following points in the order shown.

1. Is the NAS switched on and started up? Check the device indicator lights. Many consumer NAS devices enter sleep mode after a period of inactivity. Before starting the download, open the NAS administration panel from the browser to verify its availability.

2. Is the NAS reachable from the network? From the Command Prompt (Windows) or Terminal (macOS/Linux): `ping 192.168.1.100` Replace with the NAS IP address. If the command receives a reply, basic network connectivity is working.

3. Is the SMB share accessible? Attempt to open the path `\\192.168.1.100\ShareName` directly from Windows File Explorer. If the operation fails, the problem lies in the NAS SMB configuration, not in FeedDownloader Pro.

4. Are write permissions correct? Manually create a file in the destination folder via the file manager. If the operation is not permitted, the

user used to access the NAS does not have write permissions on that share. Configure the permissions from the NAS administration panel.

5. Is the firewall blocking SMB connections? The SMB protocol uses port 445 (and in some cases port 139). Verify that the system or third-party firewall is not blocking these ports for connections on the local network.

7.6 Optimal Performance on NAS

Downloads to NAS present an additional complexity compared to those to a local drive: files are written across the network and speed depends on both the LAN bandwidth and the NAS write capacity.

Operational guidance:

- **Use a wired (Ethernet) connection:** Wi-Fi introduces latency and instability in network write operations. For large archives, a wired Gigabit Ethernet connection offers significantly better performance.
- **Reduce parallel threads:** Simultaneously writing many files to a NAS can saturate its I/O. Using 2–3 parallel threads often yields better results than using the maximum available number.
- **Avoid overlapping with NAS backups:** If the NAS runs automatic backups, avoid starting batch downloads during the same time windows, as competition for the disk's I/O slows down both operations.
- **Use a local cache:** For very large archives, it is possible to download first to a fast local drive and move the files to the NAS once the download is complete.

Go to Chapter 8 for the configuration of the rename template and metadata features.

Chapter 8: File Organisation, Templates, and Metadata

8.1 The Problem of Non-Meaningful Names

When an audio file is published on a podcast server, its original name is often difficult to read: `ep_2024_03_15_FINAL_v2_mixdown.mp3`, `podcast-episode-187-compressed.m4a`, or even just `abc123def456.mp3` are common examples. These names make sense for the producer's systems, but make an archive difficult to browse.

FeedDownloader Pro resolves this problem through the **rename template system**: a mechanism that allows you to define a custom name format for all downloaded files, using information extracted directly from the RSS feed.

8.2 How the Template Works

A rename template is a text string that can contain fixed text and **tokens** — variables enclosed in single curly braces (`{ }`). Upon completion of each download, the software replaces each token with the corresponding value for the episode.

Example:

Configured template: `{date} - {podcast} - {title}`

Result: `2024-03-15 - Mario's Podcast - Episode 187: Artificial Intelligence Explained.mp3`

The file extension (`.mp3` , `.m4a` , etc.) is added automatically based on the format of the original file: it is not part of the template.

8.3 Available Tokens

Token	Description	Example
`{title}`	Episode title from the RSS feed	`Episode 187: AI Explained`
`{podcast}`	Podcast name (RSS channel title)	`Mario's Podcast`
`{date}`	Publication date in `YYYY-MM-DD` format	`2024-03-15`
`{year}`	Publication year	`2024`
`{month}`	Publication month (2 digits)	`03`
`{day}`	Publication day (2 digits)	`15`

Note: If text is entered in the template within curly braces that does not correspond to any of the listed tokens (for example `{episode}`), the text is left unchanged in the resulting file name.

8.4 Recommended Templates

Default template: `{title}` The default template uses the episode title alone. It is suitable for catalogues with descriptive titles.

For general use (recommended): `{date} - {title}` Result: `2024-03-15 - Episode 187: AI Explained.mp3`

This format is recommended because alphabetical sorting of the files coincides with chronological ordering.

For multi-podcast archives (shared folder): `{podcast} - {date} - {title}` Result: `Mario's Podcast - 2024-03-15 - Episode 187.mp3`

Useful when all podcasts are saved in the same destination folder.

For organisation into subfolders by year and month: `{year}/{month}/{date} - {title}` Creates an automatic subfolder structure (see section 8.7).

8.5 Automatic Name Normalisation

Some characters are not permitted in file names on the main operating systems: `/`, `\`, `:`, `*`, `?`, `"`, `<`, `>`, `|` on Windows.

FeedDownloader Pro automatically applies **normalisation** to the name produced by the template:

- Non-permitted characters are replaced with a hyphen (`-`) or removed.
- Double spaces are reduced to a single space.

- Leading and trailing hyphens or spaces are removed.
- The name is truncated to 240 characters if it exceeds the filesystem limit.

Note on long titles: Some podcasts use very descriptive titles (over 150 characters). Using the `{title}` token in the template may produce very long file names. In these cases, pairing `{date}` as the primary chronological element can limit the overall length of the name.

8.6 Configuring the Template

The rename template is configured in **Settings** → **Metadata**.

The text field accepts any combination of text and tokens. Beneath the field, a real-time preview is available showing the result of the template applied to a sample episode, to verify the format before saving.

The default template is `{title}`.

8.7 Organisation into Subfolders

In the template it is possible to use the `/` character to create an automatic **subfolder** structure within the destination folder.

Example — organisation by year and month: `{year}/{month}/{date} - {title}`

With a destination folder of `D:\Podcast Archive\Mario's Podcast\`, the result will be: `D:\Podcast Archive\Mario's Podcast\ ├── 2024\ │ ├── 01\ │ │ ├── 2024-01-08 - First Episode of the Year.mp3 │ │ └── 2024-01-22 - Second Episode.mp3 │ └── 03\ │ └── 2024-03-15 - Episode 187.mp3 └── 2023\ └── 12\ └── 2023-12-20 - Last Episode of 2023.mp3`

Subfolders are created automatically if they do not exist.

Caution: The `\` (backslash) character is not supported as a path separator in the template. Always use `/` (forward slash), which the software correctly translates for the operating system in use.

8.8 Sidecar JSON Files

The **"Sidecar .json File"** toggle is available in the **Settings → Metadata** tab.

When enabled, for each downloaded audio file a `.json` file is created with the same name in the same folder. The file contains the episode metadata in structured format:

```
{
  "title": "Episode 187: AI Explained",
  "podcast": "Mario's Podcast",
  "date": "2024-03-15",
  "sourceUrl": "https://media.example.com/ep187.mp3"
}
```

Use cases:

- Integration with automation scripts or systems that read metadata directly from the filesystem without querying the database.
- Preserving metadata independently of the database, useful in the event of archive migration or reconstruction.

This option is disabled by default.

8.9 ID3 Tagging

The **"ID3 Tagging"** toggle is available in the **Settings → Metadata** tab.

When enabled, upon completion of each download the software writes metadata directly into the `.mp3` file, in the standard ID3 tags:

- **Title:** The episode title
- **Artist:** The podcast name
- **Year:** The publication year
- **Cover art:** The podcast image (if available in the RSS feed)

ID3 tags are recognised by major audio players (Windows Media Player, VLC, iTunes, Foobar2000) and allow episode information to be displayed independently of the file name.

Note: ID3 tagging applies exclusively to `.mp3` files. Files in other formats (`.m4a` , `.ogg` , `.opus`) are not modified, even with this option active.

This option is disabled by default.

Go to Chapter 9 for integrity verification and archive management.

Chapter 9: Integrity, Statistics, and Archiving

9.1 Why Verify File Integrity

The completion of a download does not guarantee that the received file is intact. A network packet lost during transfer, a disk write error, or an interruption in the last second can produce a file that is formally "present" but corrupted. Without an explicit verification, an apparently complete archive can contain unplayable audio files whose corruption is only detected during playback.

FeedDownloader Pro addresses this problem with two complementary mechanisms: **size verification** (during download) and **SHA-256 verification** (upon completion).

9.2 SHA-256 Verification

SHA-256 (Secure Hash Algorithm 256-bit) is a cryptographic function that produces a 64-character hexadecimal fingerprint for any file. Two identical files always produce the same hash; a difference of even a single bit produces a completely different hash.

For each downloaded file, FeedDownloader Pro:

1. Calculates the SHA-256 hash of the file at the end of the download.

2. Saves the hash in the database, together with the file path and the date of calculation.
3. If the RSS feed includes a reference hash (some modern feeds include the `<podcast:integrity>` field), it compares it with the calculated one. In the event of a discrepancy, the file is marked as **"Corrupted"** and re-queued for a new download.

Practical uses:

- It is possible to verify at any future point that a file has not been modified, corrupted, or replaced: simply recalculate the hash and compare it with the one recorded in the database.
- After moving files to a new drive or following a migration, the **Health Check** (see section 9.4) allows you to verify that all files are still present.
- In professional contexts, the SHA-256 hash constitutes a verifiable reference of content integrity at the time of download.

9.3 Extracted Audio Metadata

Upon completion of each download, FeedDownloader Pro automatically extracts the **technical metadata** from the audio file. This information is read directly from the file (not from the RSS feed) and recorded in the database.

Extracted metadata:

Field	Description	Example
Bitrate	Audio quality in kilobits per second	`128 kbps`, `320 kbps`
Sample rate	Sampling frequency	`44100 Hz`, `48000 Hz`
Size on disk	Actual size of the downloaded file	`67.4 MB`

These values are recorded in the database and are included in the CSV export (see section 9.6).

9.4 Health Check: Archive Integrity Verification

Over time, an archive may undergo external modifications outside the software: files moved or deleted directly from the filesystem. The **Health Check** verifies the state of the archive against what is recorded in the database.

How to run the Health Check: Go to **Settings** → **Archive** → **Health Check** and click **"Start Check"**.

The process analyses each file recorded in the database and verifies that the file still exists at the recorded path. Upon completion, a summary is shown with three indicators:

Indicator	Meaning
Total	Total number of episodes in the database

Indicator	Meaning
Present	Files that exist at the recorded path
Missing	Files not found at the recorded path

The screen also shows the **total disk space** occupied by the present files.

If missing files are found, the software lists the first 5 with the podcast name and file name. To recover a missing file, use the **"Force Re-Download"** function available from the episode context menu in the main list.

9.5 Archive Statistics

The statistics section is accessible from **Settings** → **Archive** and provides a concise overview of the data recorded in the database:

- **Downloaded files:** Total number of episodes present in the database.
- **Podcasts:** Number of distinct feeds represented in the archive.
- **Time range:** Date of the first and last downloaded episode.

Statistics are automatically updated each time the Settings panel is opened.

9.6 CSV Export

The CSV export generates a file with the data of each episode present in the database. It is useful for integrating FeedDownloader Pro with other tools (spreadsheets, content management systems, automation scripts).

How to export: Go to **Settings** → **Archive** → **Export CSV** and choose the path in which to save the file.

Export columns:

Column	Content
`Podcast`	Podcast name
`Episode Title`	Episode title
`Publish Date`	Publication date
`Downloaded At`	Date and time of download
`File Size (bytes)`	File size in bytes
`Bitrate (kbps)`	Audio bitrate in kilobits per second
`Sample Rate (Hz)`	Sampling frequency in hertz
`SHA-256 Checksum`	SHA-256 hash of the file
`Validation Status`	Result of the last integrity check
`GUID`	Unique identifier of the episode in the RSS feed

File format: CSV with comma separator (`,`), UTF-8 encoding with BOM (for compatibility with Microsoft Excel). Fields containing commas are enclosed in quotation marks.

9.7 Archive Migration

To move the archive to a new drive or a new folder, use the built-in migration function, which keeps the database synchronised with the new file location.

Procedure:

1. Go to **Settings** → **Archive** → **Migrate Archive**.
2. Select the **new destination folder** via the selection window.
3. The software physically moves all audio files to the new folder and updates the paths in the database.
4. Upon completion, a summary is shown: number of files moved and any errors.

Caution: The migration moves files from the current folder to the new one. Files are removed from the original location. Verify that the destination drive has sufficient space before starting the operation.

Moving to a new computer: Copy both the audio files folder and the `feeddownloader.db` file (from the user data folder described in Chapter 2). On the new computer, install FeedDownloader Pro, copy the database to the user data folder, and use the migration function if the archive path has changed.

Go to Chapter 10 for the advanced software settings.

Chapter 10: Advanced Settings

10.1 Overview of the Settings Panel

The settings panel is accessible at any time via the gear icon (⚙) in the upper corner of the interface. Settings are organised into five thematic tabs: **General**, **Download**, **Metadata**, **Archive**, and **Advanced**. All changes are saved automatically: it is not necessary to confirm with a dedicated button.

10.2 Download

This section contains the main controls for the download engine. Internal technical parameters (connection timeout, number of retries, stall detection) are fixed in the engine and do not require manual configuration.

Parallel Threads

The number of simultaneous downloads. Selectable from three presets: **1**, **3**, and **5**. For guidelines on choosing the value, see Chapter 6.

Default value: 3

Speed Limit

Allows you to limit the aggregate bandwidth used by all active downloads, to avoid interference with other network activities.

Available values: `0` = no limit (default); any positive value in KB/s. Example: `500` limits total consumption to approximately 4 Mbps.

10.3 Keyword Filter

The text filter allows you to **narrow the list of displayed episodes** based on text contained in the title. It is a consultation and quick selection tool, particularly useful with large catalogues.

How to use the filter: The filter bar is positioned at the top of the episode list, immediately below the batch controls. By typing one or more terms, the list updates in real time showing only the episodes whose title contains **all the entered terms** (AND logic).

- To search for episodes containing the word "interview", type `interview`.
- To search for episodes containing both "interview" and "2024", type `interview 2024`.
- The filter is not case-sensitive: `Bonus` and `bonus` produce the same result.

Typical use cases:

- Quickly identifying episodes from a specific season in an extensive catalogue.

- Selecting a subset of episodes to download without scrolling through the entire list.
- Verifying whether an episode with a particular title is already present in the database.

Note: The filter acts on the display of the current list and does not modify the download queue or the status of episodes in the database. To remove the filter, clear the text bar.

10.4 General

Language

FeedDownloader Pro is available in 8 languages: Italian, English, Deutsch, Español, Français, Português, Русский, 中文.

The language change is immediate: the interface updates without needing to restart the software. The application uses exclusively the dark "Obsidian Command" theme: no light theme or list density selector is available.

10.5 Security: The 5-Level Anti-SSRF System

This section is documented for informational purposes: the security system operates in a completely automatic manner and does not require any configuration from the user.

What is an SSRF attack? SSRF (Server-Side Request Forgery) is a type of attack in which a malicious URL, instead of pointing to a public resource, points to internal network resources (such as the router administration panel, a NAS, or a local server). In the context of a downloader, a carefully crafted RSS feed could include audio URLs pointing to these internal resources.

The 5 validation levels:

1. **Protocol validation:** Only `http://` and `https://` protocols are accepted. Protocols such as `file://`, `ftp://`, `data:`, `javascript:` are rejected immediately.

2. **URL syntactic validation:** The URL is analysed to verify conformance with the RFC 3986 standard.

3. **DNS resolution with IP inspection:** The domain in the URL is resolved to an IP address. If resolution fails, the URL is rejected. If resolution succeeds, the resulting IP address is verified at the next level.

4. **Blocking of private and reserved IP addresses:** All IP addresses belonging to private or reserved ranges are blocked, including:

   - `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16` (RFC 1918 private networks)
   - `127.0.0.0/8` (loopback)
   - `169.254.0.0/16` (link-local)
   - `::1/128` (IPv6 loopback)
   - `fc00::/7` (IPv6 unique local)
   - Any address pointing to the local host.

5. **Blocking of non-standard ports:** Only ports 80 and 443 are accepted. URLs with non-standard ports (e.g. `:8080`, `:3000`, `:22`) are rejected.

Note for corporate environments: If the corporate network includes internal podcast servers reachable via private IP addresses, the anti-SSRF system will block these URLs. In this case, contact support for a custom configuration that includes specific IP address ranges in the internal whitelist.

10.6 Advanced

Updates

FeedDownloader Pro includes an integrated automatic update system.

Automatic check on startup: In the installed version (package), the software automatically checks for the availability of new updates 3 seconds after startup, by querying the GitHub repository. If a new version is available, the download begins in the background without requiring any action.

Manual check: The **"Check for Updates"** button in the **Advanced** tab forces an immediate check at any time.

If a new version is available, the software downloads it in the background and shows the **"Install and Restart"** button. The installation is never started automatically: the decision always rests with the user.

States displayed during the process:

- **Checking for updates...** — the software is querying the GitHub repository.
- **Up to date** — the installed version is the most recent.

- **New version available (vX.Y.Z)** — background download in progress.
- **Update ready** — the package has been downloaded and is ready for installation.

Reset Database

Completely deletes the database and starts again with an empty archive. **This operation is irreversible.** The software requires explicit double confirmation before proceeding. Audio files on disk are not deleted: only the software's internal memory is cleared (download history, metadata, statistics).

When to use it: Only when you intend to start from a completely empty archive, for example after migrating to a new system or to remove data from a test cycle.

Go to Chapter 11 for troubleshooting the most common problems.

Chapter 11: Troubleshooting

11.1 How to Use This Chapter

This chapter collects the most common problems reported by users, with the most likely causes and step-by-step solutions. Each problem is described in the way it manifests in the interface, not in internal technical terms.

If the problem is not in this list, consult the log files in the `logs/` folder (see Chapter 10) and contact support attaching the log from the session in which the problem occurred.

Feed and Analysis Problems

Problem: "Connection error" or "Timeout" during feed analysis

How it manifests: You click **"Analyse"** and after a few seconds an error message appears indicating a timeout or connection failure. The list remains empty.

Likely causes and solutions:

- **The feed server is unavailable.** Open the feed URL in the browser. If the browser returns an error (page not found, "This site can't be

reached"), the problem lies with the podcast server: nothing can be done except retrying later.

- **The internet connection is unavailable or unstable.** Verify that other websites are reachable. If the connection is unstable, wait for it to stabilise before retrying.
- **A corporate firewall or proxy is blocking the request.** In corporate environments, traffic to certain hosts may be blocked. Try from a home network to verify whether the problem is specific to the corporate network.

Problem: The feed is analysed but the episode list is empty

How it manifests: The analysis completes without errors, but the episode list shows no items (or shows 0 episodes).

Likely causes and solutions:

- **The feed contains no episodes.** Open the URL in the browser and verify that the XML document contains `<item>` or `<entry>` tags. If they are not present, the podcast has not yet published any episodes.
- **The feed uses a non-standard format.** FeedDownloader Pro supports RSS 2.0 and Atom 1.0. Some feeds produced by proprietary platforms may have an unconventional structure. In this case, the software displays a specific warning in the analysis message.
- **All episodes are already in the database.** If the feed has been analysed previously, episodes appear with **"Downloaded"** status (muted green). Scroll through the list and check for this status indicator.

Problem: The feed shows only the last N episodes and not the full historical catalogue

How it manifests: You analyse a podcast with hundreds of known episodes, but the list shows only 50 or 100.

Cause: This limit is imposed by the podcast publisher or its hosting platform, not by FeedDownloader Pro. Many platforms limit the RSS feed to the last 50–100 episodes to reduce the load on their servers. The software downloads exactly the data that the feed makes available.

Possible alternatives:

- Check whether the podcast offers a "full feed" as an alternative URL (some platforms make this available).
- Consult the podcast's website or distribution platform (Spotify, Apple Podcasts) to retrieve links for older episodes.
- Some platforms accept parameters in the URL to request the complete feed (e.g. `?limit=0` or `?paged=all`): check the documentation of the specific platform.

Download Problems

Problem: Many episodes show "Error 404" status

How it manifests: After a batch download, numerous episodes show **"Error"** status with the message `404 Not Found`.

Cause: The episodes are still present in the RSS feed (in the XML document), but the audio files they point to have been removed from the server. This situation is common with abandoned podcasts or those migrated to other platforms.

What can be done:

- It is not possible to download files that no longer exist on the server.
- If this is an active podcast and the errors seem excessive, contact the podcast publisher: it may be a temporary migration or a resolvable technical issue.
- Episodes with 404 errors are automatically excluded from subsequent batches. It is not necessary to remove them from the list.

Problem: Downloads start but proceed very slowly

How it manifests: The progress bar moves, but the speed is very low (a few KB/s) relative to the available bandwidth.

Likely causes and solutions:

- **The podcast server applies bandwidth limitations.** Many hosting servers impose throttling to contain costs. Reducing threads to 1 may improve the situation with servers that penalise multiple connections.
- **The Wi-Fi connection is unstable.** For intensive batch downloads, use a wired (Ethernet) connection.
- **The destination drive is slow.** Writing to a NAS over a Wi-Fi connection or to USB 2.0 devices can be the bottleneck. Consider downloading to a fast local drive first.

- **The internet connection is genuinely limited.** Check the actual download speed with a speed test. If the result is below expectations, the problem lies with the connection.

Problem: An episode remains stuck at a high percentage and never completes

How it manifests: A single download shows a high percentage (90%, 95%, 99%) that never reaches 100% and does not update.

Cause: The server sent almost all of the file but interrupted the transfer before completion. The stall detection will detect this condition within 60 seconds of the last data received and will automatically restart the download.

If the problem persists after multiple attempts: The file on the server may be corrupted or truncated. After the maximum number of attempts, the episode will be marked as **"Error"** with a message indicating a discrepancy between the declared size and the received size.

Problem: The software downloaded an `.mp3` file but the audio player reports it as corrupted

How it manifests: The download shows as completed (green status), but opening the file with an audio player returns an error or the file does not play.

Cause: This should not occur thanks to the `.part` file mechanism and size verification. If it does happen, the original file on the server may already be corrupted (a publisher issue), or a disk write error occurred.

Solution:

1. Right-click on the episode in the list → **"Force Re-Download"**.
2. If the re-downloaded file is still corrupted, the problem lies with the source file on the podcast server. Verify this by opening the file URL directly in the browser.
3. Run a Health Check (see Chapter 9) to verify whether other files in the archive have problems.

NAS and Network Problems

Problem: "Network path not reachable" even though the NAS is switched on

How it manifests: The software shows the path not reachable warning, but the NAS is accessible normally from the file manager.

Solutions to check in order:

1. **Verify that the path is exact.** A difference in capitalisation (`\\MYNAS\podcast` vs `\\MYNAS\Podcast`) can cause an error on some systems.
2. **Are the SMB credentials stored?** Open File Explorer and attempt to access `\\MYNAS\ShareName` manually. If a password is requested, the credentials are not saved in the Windows Credential Manager. Enter them and tick **"Remember"**.

3. **Is the Windows Firewall blocking FeedDownloader Pro?** Go to `Control Panel → Windows Defender Firewall → Allowed apps` and verify that FeedDownloader Pro is listed with access permitted.
4. **Does the NAS support SMBv2/3?** Some older NAS devices support only SMBv1, which is disabled by default on Windows 11. Update the NAS firmware or enable SMBv1 from the NAS administration panel.

Problem: Downloads to NAS stop after a few minutes

How it manifests: The batch starts normally, downloads a few episodes, then stalls with write errors or a path not reachable error.

Cause: The NAS enters sleep mode during the download. Some consumer NAS devices have a power-saving function that can activate even during active transfers if the device is configured to monitor only web traffic, ignoring SMB connections.

Solutions:

- Temporarily disable sleep mode from the NAS administration panel during batch downloads.
- Reduce the number of threads to 1: a continuous write stream prevents sleep activation more effectively than intensive bursts with intermediate pauses.

General Problems

Problem: The interface responds with a delay

How it manifests: Clicks take 1–2 seconds to respond, scrolling through the list is jerky, the programme appears slow.

Likely causes:

- **Large database.** With tens of thousands of episodes in the database, some operations may slow down. Consider using **Reset Database** (**Settings → Advanced**) only if the archive contains many episodes in error status or data that you do not intend to recover.
- **High number of threads on hardware with limited RAM.** With 5 active threads on a system with less than 4 GB of RAM, the process may be slow. Reduce threads to 1 or 3.
- **Antivirus scanning `.part` files in real time.** Some security software intercepts every disk write operation, slowing downloads. Add the destination folder to the antivirus exclusions.

Problem: The software does not start or closes immediately upon opening

How it manifests: The programme is started, the process briefly appears in the Task Manager but then disappears without the interface being displayed.

Solutions:

1. **Check the logs.** Access the `%APPDATA%\FeedDownloaderPro\logs\` folder (Windows) or `~/.config/FeedDownloaderPro/logs/` (Linux). Open the most recent log file with a text editor: the last line should indicate the cause of the problem.

2. **Corrupted database.** If the log indicates a SQLite error on startup, the `feeddownloader.db` file may be corrupted. Replace it with a backup (see Chapter 9). If no backup is available, rename it to `feeddownloader.db.bak`: the software will create a new empty database on the next startup (with loss of history).
3. **Reinstall the software.** Uninstall FeedDownloader Pro and install the most recent version. The database and settings are not deleted by the uninstallation.

Problem: I have lost my database data — is it possible to recover it?

How it manifests: The database has been accidentally deleted, is corrupted, or a reset was performed without a prior backup.

Recovery possibilities:

- **With a backup available:** Copy the backup `feeddownloader.db` file to the application's user data folder, with the programme closed (see Chapter 2 for the user data folder path).
- **Without a backup:** The audio files on disk are still present: only the software's memory has been lost. It is possible to partially reconstruct the archive by re-analysing the feeds: episodes whose files are already present on disk will be recognised by the system and will not be re-downloaded.
- **Prevention:** Periodically make a manual copy of the `feeddownloader.db` file to a safe location, or export the feed list in OPML format (see Chapter 5) as a configuration backup. It is advisable to perform this backup before any migration or software update.

This is the final chapter of the Runtime FeedDownloader Pro Advanced User Manual.

For assistance not covered by this manual, refer to the official releases page or contact the technical support of Ecosystem Runtime | Digital Core.

Ecosystem Runtime | Digital Core — Tools built to last.